

turboswarm: A general-purpose, extensible Particle Swarm Optimization library with a Rust core

Jose L. Salmeron*

June 2026

Abstract

turboswarm is a general-purpose, open-source Particle Swarm Optimization (PSO) library with a compute core written in Rust and an ergonomic Python API. It targets real, integer, mixed, binary and interval-valued (“grey”) decision variables, and exposes established algorithmic variants, swarm topologies, constraint handling, multi-objective optimization, and visualization behind a single, uniform interface. The optimization loop is fully decoupled from any concrete variant through three traits (**SearchSpace**, **Velocity**, **Topology**), so adding a new variant is a self-contained trait implementation rather than a fork of the core. The same Rust core is usable directly from Rust through zero-cost generics and from Python through PyO3 and **maturin**, where native objectives run without holding the Global Interpreter Lock. Every experiment is seedable and therefore reproducible. On a five-function benchmark across three dimensions, the compiled core is the fastest among the libraries compared, with a measured speedup of roughly 2–3.5× over the fastest pure-Python competitor at comparable solution quality. The library is distributed on PyPI (**turboswarm**) and crates.io (**turboswarm-core**), documented, and archived on Zenodo (DOI 10.5281/zenodo.20832446).

1 Introduction

Particle Swarm Optimization [5] is a population-based, gradient-free optimization method in which a swarm of candidate solutions (“particles”) moves through the search space guided by each particle’s own best position and the best position found in its neighbourhood. It is one of the most widely used metaheuristics for continuous and combinatorial optimization, with applications across engineering, machine learning and the sciences.

Researchers and students need a PSO implementation that is fast, reproducible, broad in scope, and easy to extend with new variants for comparative studies. Existing Python libraries cover parts of this need, but they are implemented predominantly in pure Python, which ties the cost of the optimization loop to the interpreter and makes large or repeated experiments slow. **turboswarm** was written to provide a compiled, reproducible and extensible PSO toolkit that is nonetheless as easy to call as a pure-Python package, suitable for research experiments, algorithm benchmarking and teaching.

2 State of the field

Several mature open-source PSO and metaheuristics packages exist, including **pyswarms** [7], **pymoo** [1], **DEAP** [4] and **nevergrad** [8]. These are valuable and widely adopted, but they are implemented

*CUNEF Universidad, Madrid, Spain. ORCID: 0000-0001-7811-3716.

mainly in pure Python, so the swarm loop itself runs in the interpreter. `turboswarm` differs in three ways that motivated building a new library rather than contributing to an existing one. First, its optimization loop and native objectives are compiled (Rust) and run without the Global Interpreter Lock (GIL), giving a measured speed advantage (Section 6). Second, it treats extensibility as a first-class design goal: a new variant is a self-contained trait implementation, not a fork of the core (Section 3). Third, it provides a grey-number (interval-valued) search space, in which each decision variable is known only to lie within an interval — a capability we are not aware of in the libraries above. Together these make `turboswarm` complementary to, rather than a re-implementation of, the existing ecosystem.

3 Software design

The optimization loop is fully decoupled from any concrete variant through three traits.

- **SearchSpace** defines the domain. The only difference between real and integer variables lives in its `decode` step, so the swarm always operates on a continuous internal representation ($\mathbb{R}^n$) and discretization happens only at evaluation time.
- **Velocity** defines the update rule, so a single PSO variant — inertia [9], constriction [2] or the Fully Informed Particle Swarm [6] — is exactly one implementation of this trait.
- **Topology** defines the social structure of the swarm: global (gbest), ring (lbest), Von Neumann or random.

Because the loop is generic over these traits, adding a variant requires no change to the core and is accompanied by a convergence test against a known optimum. For the Python boundary the same traits are implemented for boxed trait objects, so variants can be selected at runtime by name without duplicating the loop. This design confines the real/integer/mixed/binary/grey distinction to the search space and makes the library a convenient base for comparing variants under identical conditions.

4 Features

- **Velocity variants:** inertia [9], constriction [2] and Fully Informed PSO [6].
- **Topologies:** global (gbest), ring (lbest), Von Neumann and random.
- **Search spaces:** continuous, integer (with a discretization strategy), binary, mixed (per-dimension type) and grey (interval-valued variables, optimized over their centre and spread).
- **Multi-objective (MOPSO):** an external Pareto archive with a turbulence operator, crowding-distance or Coello’s adaptive grid [3] for diversity, and a hypervolume quality indicator.
- **Constraints:** inequality and equality constraints via a penalty, plus an optional repair operator that maps candidates back to the feasible set.
- **Run control:** stopping on a target value, evaluation budget, wall-clock budget or stagnation, with a per-iteration callback and a reported stop reason.
- **Boundary handling:** clamp, reflect, wrap or reinitialize.

- **Performance:** velocity clamping, `rayon`-based parallel evaluation and a NumPy-vectorized batch path.
- **Hyperparameter sensitivity:** a Cartesian-product sweep over hyperparameter values, aggregated over seeds.
- **Benchmarks:** sphere, rastrigin, rosenbrock, ackley, griewank, schwefel and the CEC-family functions (bent cigar, discus, elliptic, zakharov, levy, expanded Schaffer), each with metadata (recommended bound, known optimum).
- **Visualization (Python):** convergence curves, variant comparison, 2D and 3D swarm animation, Pareto-front and sensitivity plots.
- **Integrations:** a SciPy `minimize` drop-in, a scikit-learn `PSOSearchCV`, an Optuna sampler, pandas export and parallel-objective backends.

5 Usage

From Python, an objective can be a callable or the name of a native benchmark; bounds are given per dimension or as a single pair broadcast over `dim`.

```
import turboswarm as pso

# Native benchmark, runs in Rust without the GIL
r = pso.minimize("rastrigin", bounds=(-5.12, 5.12), dim=2, seed=42)

# A user objective, a chosen variant and topology
r = pso.minimize(lambda x: sum(xi*xi for xi in x),
                 bounds=[(-5, 5)] * 3, velocity="fips", topology="ring", seed=1)
print(r.best_position, r.best_value)
```

The same core is callable from Rust through zero-cost generics:

```
use turboswarm_core::prelude::*;
use turboswarm_core::benchmarks::rastrigin;

let space = ContinuousSpace::uniform(2, -5.12, 5.12);
let velocity = InertiaVelocity::new(0.9, 1.49445, 1.49445).with_decay(0.4);
let params = PsoParams { seed: Some(42), ..Default::default() };
let result = Pso::new(space, velocity, GlobalBest::new(), params).minimize(rastrigin);
```

6 Performance

To illustrate the benefit of the compiled core we compare `turboswarm` against four widely used baselines — `pyswarms` [7], `pyswarm`, `pymoo` [1] and DEAP [4] — on five standard functions (sphere, rastrigin, ackley, rosenbrock, griewank) across three dimensions (2, 10, 30), using an identical swarm configuration for every library (40 particles, 200 iterations, $w = 0.729$, $c_1 = c_2 = 1.49445$; median over five seeds after a warm-up). DEAP, a general-purpose evolutionary-computation toolkit, provides a PSO *recipe* rather than a packaged solver, which we adapted to the same inertia rule and configuration. We report both `turboswarm`’s native objective path (objective evaluated in Rust) and its Python-callable path (objective evaluated in Python).

Table 1 reports the median wall-clock time over the five functions at each dimension, and Table 2 the resulting speedup of the native path over every baseline. Through its native path `turboswarm` is the fastest library at every dimension: about $3.2\times$ faster than `pyswarms` (the fastest competitor) at two dimensions, narrowing to $2.1\times$ at thirty as `pyswarms`’ vectorized NumPy objective amortizes more of its per-iteration overhead. Against the interpreter-bound, non-vectorized baselines the gap is larger and persistent: a median of about $12\times$ over DEAP, $7\text{--}14\times$ over `pyswarm`, and $24\text{--}70\times$ over `pymoo`. Even `turboswarm`’s own Python-callable path is competitive with the fastest baseline at low dimension. Solution quality was comparable across all libraries — the median best objective values fell within the same order of magnitude (for example $0.08\text{--}0.11$ at ten dimensions) — so the speed advantage does not come at a cost in solution quality.

Table 1: Median wall-clock time (ms) over the five benchmark functions, by dimension. Lower is better.

Library	$d = 2$	$d = 10$	$d = 30$
<code>turboswarm</code> (native)	3.23	5.25	9.85
<code>turboswarm</code> (Python)	43.87	50.44	58.48
<code>pyswarms</code>	10.43	13.40	20.52
<code>pyswarm</code>	45.61	64.81	67.41
DEAP	46.01	64.09	106.55
<code>pymoo</code>	225.89	227.38	241.71

Table 2: Speedup of `turboswarm` (native path) over each baseline (median time ratio over the five functions). Higher is better.

Baseline	$d = 2$	$d = 10$	$d = 30$
<code>turboswarm</code> (Python)	$13.6\times$	$9.6\times$	$5.9\times$
<code>pyswarms</code>	$3.2\times$	$2.6\times$	$2.1\times$
<code>pyswarm</code>	$14.1\times$	$12.3\times$	$6.8\times$
DEAP	$14.2\times$	$12.2\times$	$10.8\times$
<code>pymoo</code>	$69.8\times$	$43.3\times$	$24.5\times$

The benchmark was run with `turboswarm` 0.2.2, `pyswarms` 1.3.0, `pyswarm` 1.0.0, `pymoo` 0.6.1.6, DEAP 1.4 and NumPy 2.4.6 under Python 3.12, on an Apple M5 (10 cores, macOS 26.5). Absolute numbers are machine-dependent; the full benchmark — including the raw results and machine provenance — is reproducible with `benches/bench_suite.py`.

7 Availability and citation

`turboswarm` is released under the MIT license. The Python package is on PyPI as `turboswarm` (`pip install turboswarm`) and the Rust core on crates.io as `turboswarm-core`. The source code, documentation and issue tracker are at <https://github.com/turboswarm/turboswarm.github.io>, with a documentation portal at <https://turboswarm.github.io>. Each release is archived on Zenodo under the concept DOI [10.5281/zenodo.20832446](https://doi.org/10.5281/zenodo.20832446). If you use `turboswarm` in academic work, please cite this paper and the archived software release.

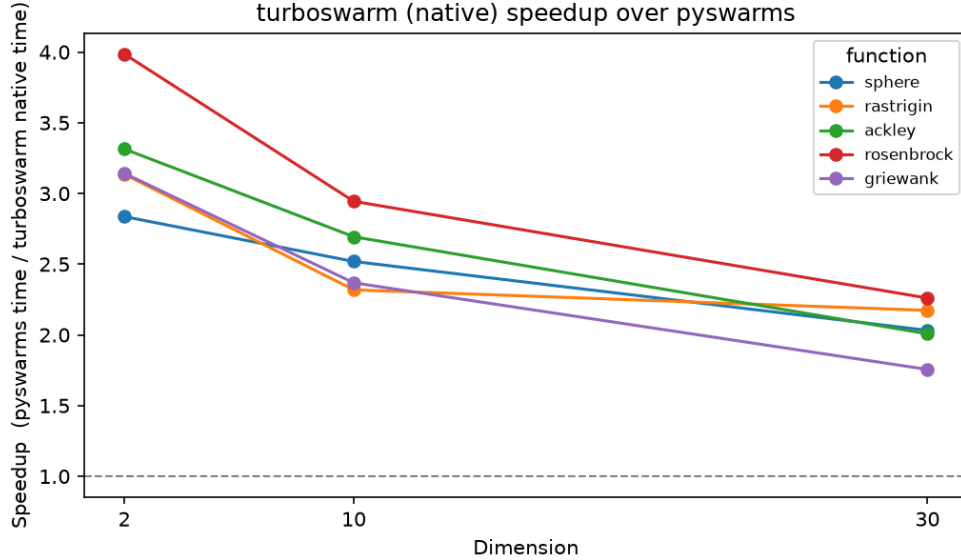


Figure 1: Speedup of `turboswarm` (native objective path) over `pyswarms` across dimensions, for five benchmark functions.

8 Conclusion

`turboswarm` provides a compiled, reproducible and extensible PSO toolkit that is as easy to call as a pure-Python package. By exposing many PSO variants, topologies and search spaces — including an interval-valued “grey” space — behind one reproducible interface, it lowers the cost of comparative algorithmic studies and supports teaching through its built-in visualization. Its trait-based core is also the foundation for ongoing methodological work on optimization with grey (interval-valued) variables.

AI usage disclosure

The `turboswarm` library itself — the Rust core, the Python bindings and the algorithms — was designed and written by the author. Generative AI (Anthropic’s Claude, used through Claude Code) assisted with drafting this paper, implementing the comparative benchmark suite, and editing parts of the documentation. All AI-assisted text and code were reviewed, tested and validated by the author, who takes full responsibility for the content.

Acknowledgements

The author received no specific funding for this work, and acknowledges the open-source Rust and PyO3 communities, whose tooling underpins the Rust–Python boundary used here.

References

- [1] Julian Blank and Kalyanmoy Deb. `pymoo`: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509, 2020. doi: 10.1109/ACCESS.2020.2990567.

- [2] Maurice Clerc and James Kennedy. The particle swarm - explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1): 58–73, 2002. doi: 10.1109/4235.985692.
- [3] Carlos A. Coello Coello, Gregorio Toscano Pulido, and Maximino Salazar Lechuga. Handling multiple objectives with particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, 8(3):256–279, 2004. doi: 10.1109/TEVC.2004.826067.
- [4] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner, Marc Parizeau, and Christian Gagné. Deap: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13:2171–2175, 2012.
- [5] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN’95 - International Conference on Neural Networks*, volume 4, pages 1942–1948. IEEE, 1995. doi: 10.1109/ICNN.1995.488968.
- [6] Rui Mendes, James Kennedy, and José Neves. The fully informed particle swarm: simpler, maybe better. *IEEE Transactions on Evolutionary Computation*, 8(3):204–210, 2004. doi: 10.1109/TEVC.2004.826074.
- [7] Lester James V. Miranda. Pyswarms: a research toolkit for particle swarm optimization in python. *Journal of Open Source Software*, 3(21):433, 2018. doi: 10.21105/joss.00433.
- [8] Jérémy Rapin and Olivier Teytaud. Nevergrad - a gradient-free optimization platform. <https://GitHub.com/FacebookResearch/Nevergrad>, 2018.
- [9] Yuhui Shi and Russell Eberhart. A modified particle swarm optimizer. In *1998 IEEE International Conference on Evolutionary Computation Proceedings*, pages 69–73. IEEE, 1998. doi: 10.1109/ICEC.1998.699146.